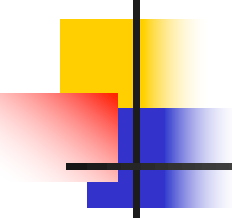


Packages

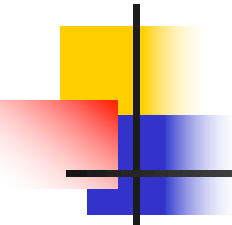


Dr. Khadim Dramé
Khadim.drame@univ-zig.sn
Département Informatique
Université Assane Seck de Ziguinchor



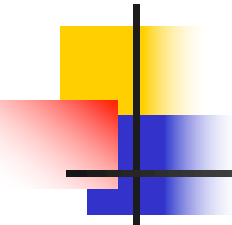
Plan du cours

- Introduction
- Définition de packages
- Utilisation de types d'un package
- Package et accessibilité
- Packages de Java SE



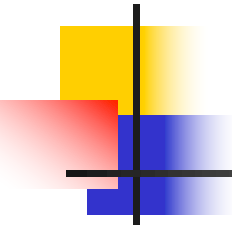
Notion de package

- Regroupement de classes et interfaces en ensembles dits **packages** (**paquetages**)
- Un **package** est constitué de classes et interfaces couvrant un **même domaine**
- Packages = bibliothèques dans d'autres langages
- Deux catégories de packages
 - Packages intégrés dans Java SE
 - Packages définis par les utilisateurs



Intérêt des packages

- Les packages offrent un niveau de modularité supplémentaire
- Les packages permettent de
 - trouver facilement l'emplacement des classes
 - éviter la confusion entre des classes de même nom
 - mieux structurer les classes
 - contrôler l'accès aux attributs et méthodes



Définition de packages

- Définition de package

```
package nom_package;
```

- Instruction à spécifier au début du fichier source

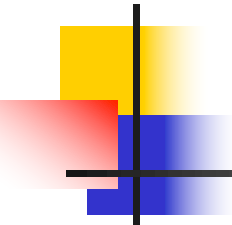
- Un package peut avoir des sous packages

```
package nom_package.nom_sous_package;
```

- Règles de nommage

- par convention, nom de package en minuscule
 - recommandation d'utiliser un nom de domaine inversé
 - un nom de package ne peut pas commencer par java ou javax

- Une classe ne peut appartenir qu'à un seul package



Utilisation de types d'un package

- Plusieurs cas pour utiliser des types (classes, interfaces) d'un package donné:

- Le type est **dans le package** `java.lang`

➡ on peut l'utiliser directement sans spécifier le package

```
String chaine = "Bonjour";
```

- Le code appelant est **dans le même package** que le type

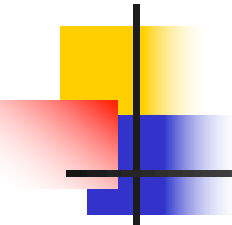
➡ on peut l'utiliser directement sans spécifier le package

- Aucun des cas précédents

➡

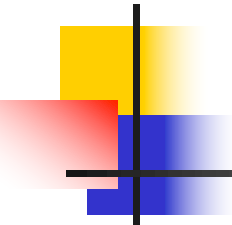
- soit importer le type
- soit spécifier le nom complet (*qualified name*) du type

```
java.util.ArrayList liste = new java.util.ArrayList();
```



Importation de types d'un package

- L'importation permet de **ne pas préfixer** un type par son package
- Elle doit se faire **après la déclaration de package** (si cette dernière est présente)
- Les classes du package **java.lang** sont importées automatiquement
- Remarque
 - Si deux packages contiennent le même nom de classe, utiliser les noms complets des classes



Importation de types d'un package

- Importation d'une classe

```
import nom_package.nom_classe;
```

```
import java.util.ArrayList;
```

- Importation de tous les types d'un package

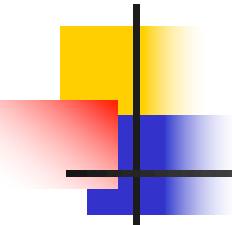
```
import nom_package.*;
```

```
import java.util.*;
```

- L'importation des types d'un package **n'entraîne pas l'importation des types des sous-packages**

```
import javax.swing.*;
```

```
import javax.swing.event.*;
```

Importation de membres statiques

- Importation possible des membres **statiques** d'une classe ou d'une interface (version 5.0 de Java SE)
- Importation d'un membre statique

```
import static nom_package.nom_member;
```

```
import static java.lang.Math.PI;
```

- Importation de tous les membre statiques

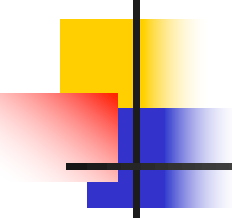
```
import static nom_package.*;
```

```
import static java.lang.Math.*;
```

```
...
```

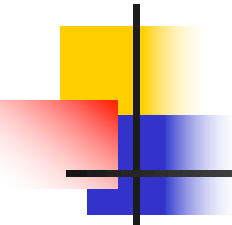
```
x = sqrt(2); // au lieu de Math.sqrt
```

```
s = 2*PI; // au lieu de Math.PI
```



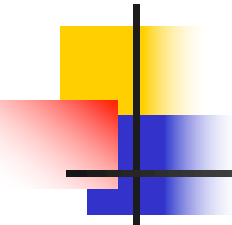
Package et accessibilité

- **Par défaut**, un type n'est **accessible** qu'aux membres du même package
 - il suffit d'omettre l'information de **portée**
 - on parle de **portée de niveau package**
- Les types d'un package, destinées à l'exportation, doivent être déclarées **publiques**
 - elles seront **accessibles partout**



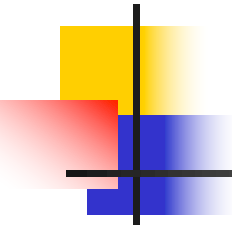
Packages de Java SE

- Quelques packages de Java SE
 - **java.lang** : classes de base de Java
 - **java.util** : classes utilitaires
 - **java.io** : gestion des flux d'entrées-sorties, fichiers
 - **java.awt** : réalisation d'interfaces graphiques
 - **javax.swing** : réalisation d'interfaces graphiques avancées
 - **java.applet** : gestion des applets
 - **java.net** : support de réseau
 - **java.rmi** : gestion des objets distribués
 - **java.sql** : gestion bases de données relationnelles



Package java.lang

- **java.lang** contient les classes de base
 - **Object**: super classe de toutes les classes Java
 - **System**: fournit des attributs et méthodes utiles (System.out.println)
 - **String**: permet de manipuler les chaînes de caractères
 - **StringBuffer**: similaire à la classe **String**
 - **Math**: assure les opérations mathématiques (cos, abs, pow, sqrt, etc.)
 - **Runtime**: permet entre autres de lancer une application extérieure à partir d'un programme Java



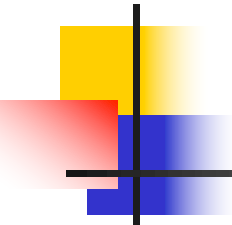
Package java.lang

■ Classe **String**

- Elle est déclarée **final** (elle est non extensible)
- Elle est **immuable** : l'état de ses instances ne changera pas après leur création

■ Classe **StringBuffer**

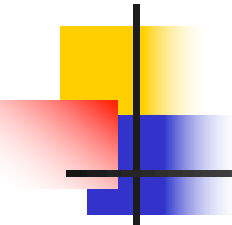
- Un objet de type StringBuffer peut être utilisé partout où un objet de type String est utilisé
- Il est simplement plus flexible : on peut modifier son contenu (**mutable**)



Package java.lang

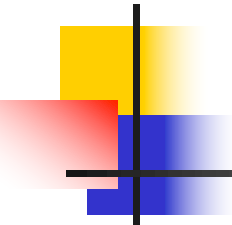
■ Exemple

```
package sn.uasz.m1.test;
public class TestString{
    public static void main (String args[]){
        String msg1 = "Bonjour la classe";
        String msg2 = "Bonjour ";
        msg2 += "la classe";
        String msg3 = "Bienvenue !";
        System.out.println(msg1==msg2);//false
        System.out.println(msg1.equals(msg2));//true
        System.out.println(msg1.equals(msg3));//false
        String ss = msg1.substring(0, 7);
        System.out.println(ss);//Bonjour
        String msg = "Bienvenue";
        msg = msg.concat(" au cours ");
        System.out.println(msg);//Bienvenue au cours
        msg += " de P00 !";
        System.out.println(msg);//Bienvenue au cours de P00 !
    }
}
```



Package java.util

- **java.util** contient un ensemble de classes et interfaces utilitaires
 - Gestion des collections
 - **List** : manipuler des tableaux de taille dynamique
 - **Deque** : manipuler des piles
 - **Map** : manipuler des tables de hachage
 - Gestion des dates
 - **Date** : manipuler des dates
 - **Calendar** : manipuler des dates et calendriers
 - **SimpleDateFormat** : spécifier le format de la date
 - Génération de nombre aléatoires
 - **Random** : générer des nombres aléatoires



Package java.util

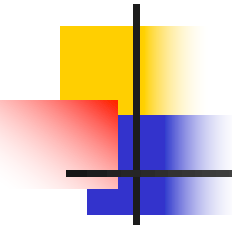
- **Scanner**: lire des données sur l'entrée standard (clavier) et contrôler leur validité

```
package sn.uasz.m1.test;
import java.util.Scanner;
public class TestScanner1{
    public static void main (String args[]){
        Scanner scanner = new Scanner(System.in);
        System.out.print("Donner votre nom : ");
        String nom = scanner.nextLine();
        System.out.print("Donner votre âge : ");
        int age = scanner.nextInt();
        System.out.println("Nom : "+nom+" et âge : "+age);
    }
}
```




Package java.util

```
package sn.uasz.m1.test;
import java.util.Scanner;
public class TestScanner2{
    public static void main (String args[]){
        Scanner scanner = new Scanner(System.in);
        System.out.print("Donner votre nom : ");
        String nom = scanner.nextLine();
        Integer age = null;
        do {
            System.out.print("Donner vore âge : ");
            if (!scanner.hasNextInt()) {
                scanner.next();
                System.out.println("Saisir une valeur valide svp ");
                continue;
            }
            age = scanner.nextInt();
        } while(age == null);
        System.out.println("Nom : "+nom+" et âge : "+age);
    }
}
```



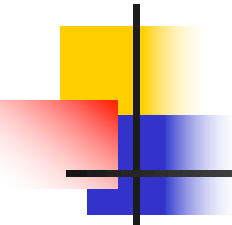
Package java.util

- Classe **Date**

- La première classe définie pour représenter une date
- Une bonne partie de ses méthodes est dépréciée mais la elle est largement utilisée

- Classe **Calendar**

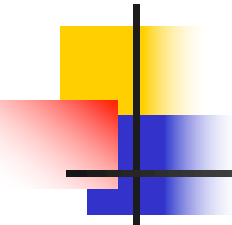
- Elle est plus complète que la classe **Date** mais reste moins utilisée



Package java.lang

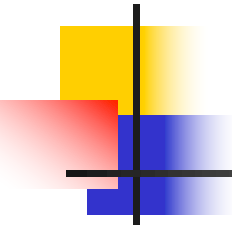
■ Exemple

```
package sn.uasz.m1.test;
import java.util.Date;
public class TestDate{
    public static void main (String args[]){
        Date date1 = new Date();
        //crée la date du jour avec l'heure actuelle
        Date date2 = new Date(0); //01/01/1970 00:00:00.000
        //date2.set(2022, 6, 10, 12, 0, 0);
        //on met la date du 10 juin 2022 à 12:00:00
        System.out.println(date1);
        System.out.println(date2);
        System.out.println(date1.after(date2));//true
        System.out.println(date2.before(date1));//true
        System.out.println(date1.equals(date2));//false
        System.out.println(date1.compareTo(date2));//1
    }
}
```



Package java.time

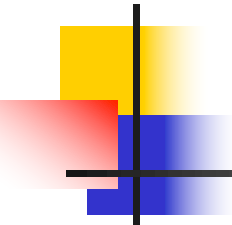
- De nouvelles classes ont été introduites par Java 8 dans le package `java.time`
 - `LocalDate` pour représenter une date
 - `LocalTime` pour représenter une heure
 - `LocalDateTime` pour représenter une date et une heure
 - `Instant` pour représenter un point dans le temps
 - `Duration` pour représenter une durée
 - `Period` pour représenter une période (durée en années, mois,...)



Package java.time

■ Exemple 1

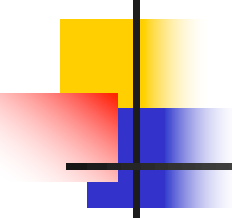
```
package sn.uasz.m1.test;
import java.time.*;
public class TestDateTime{
    public static void main (String args[]){
        LocalDate dateActuel = LocalDate.now();
        LocalTime heureActuel = LocalTime.now();
        LocalDateTime dateHeureActuel = LocalDateTime.now();
        LocalDate date = LocalDate.of(2022, Month.JUNE, 12);
        //12/06/2022
        LocalDateTime dateTime = date.atTime(12, 00);
        //12/06/2022 12:00:00
        LocalTime time = dateTime.toLocalTime();//12:00:00
        System.out.println(date);//2022-06-12
        System.out.println(time);//12:00
    }
}
```



Package java.time

■ Exemple 2

```
package sn.uasz.m1.test;
import java.time.*;
public class TestDateTime{
    public static void main (String args[]){
        Instant debut = Instant.now();
        // traitement à mesurer
        Instant fin = Instant.now();
        Duration duree = Duration.between(debut, fin);
        System.out.println(duree.toMillis());
        LocalDate dateDebut = LocalDate.of(2020, 05, 25);
        LocalDate dateFin = LocalDate.of(2022, 10, 15);
        Period period = Period.between(dateDebut, dateFin);
        System.out.println(period);//P2Y4M20D
        System.out.println(period.getYears());//2
        System.out.println(period.getMonths());//4
        System.out.println(period.getDays());//20
    }
}
```



Exercice d'application

- Créer une classe **Employe** caractérisée par un matricule, un nom, une date de naissance et une date d'embauche.
- Définir les méthodes *getAge()* et *getAnciennete()* permettant de calculer et retourner respectivement son âge et son ancienneté.